INFORME EV-2

Infraestructura Docker y análisis de vulnerabilidades

Gestión de Seguridad de la Información

Integrantes: Leslie Cáceres · Vicente Falcón

Docente: Héctor Tejo

Fecha: miércoles 28 de octubre de 2026

Índice

Resumen .....
1. Introducción .....
2. Marco conceptual: Docker .....
    2.1 Docker .....
    2.2 Imagen .....
    2.3 Dockerfile .....
    2.4 Volumen .....
    2.5 Red en Docker .....
    2.6 Docker Compose .....
3. Arquitectura .....
4. La imagen propia .....
5. La aplicación .....
    Roles y alcance .....
    Funcionalidad .....
    Modelo de datos .....
    Capturas .....
6. Escaneo con OWASP ZAP .....
    Escáner 1: pasivo .....
    Escáner 2: activo .....
    El límite del escáner automático .....
7. Escaneo con OpenVAS .....
    Escáner 1: reconocimiento .....
    Escáner 2: completo .....
8. Tabla de vulnerabilidades .....
9. Plan de mitigación .....
10. Conclusiones .....
Referencias .....

Resumen

El trabajo monta una infraestructura de cuatro contenedores Docker sobre una red propia: una base de datos Oracle, una aplicación web bancaria construida con una imagen propia, y dos escáneres de vulnerabilidades. La aplicación se comunicó con la base entre contenedores; los escáneres la analizaron desde dentro de la misma red. El resultado se consolidó en una tabla de vulnerabilidades y en un plan de mitigación.

1. Introducción

La evaluación consiste en desplegar una infraestructura con Docker formada por cuatro contenedores y analizarla con OWASP ZAP y OpenVAS. En este informe se documentan los seis conceptos solicitados, la arquitectura, la imagen propia construida con Dockerfile, los escaneos realizados, las vulnerabilidades detectadas y el plan de mitigación correspondiente.

El proyecto tiene cuatro contenedores. Dos sostienen el servicio (la base de datos y la aplicación) y dos lo analizan: ZAP para la capa web y OpenVAS para la infraestructura. Los cuatro corren en la red `soc-net`, de modo que se hablan entre sí sin salir al exterior.

2. Marco conceptual: Docker

Las definiciones que siguen van en el orden en que se usan al levantar el proyecto.

2.1 Docker

Docker es una plataforma que empaqueta una aplicación junto con todo lo que necesita para ejecutarse (bibliotecas, variables de entorno, sistema base) dentro de una unidad llamada contenedor. La diferencia con una máquina virtual está en el núcleo: la máquina virtual trae su propio sistema operativo completo, mientras que el contenedor comparte el núcleo del equipo anfitrión y solo carga lo que le falta. Eso lo hace mucho más liviano y rápido de arrancar.

En este proyecto, Docker Desktop sobre Linux entrega el motor y la interfaz de línea de comandos con la que se levantó todo.

2.2 Imagen

Una imagen es la plantilla de solo lectura desde la que se crea un contenedor. Contiene el sistema de archivos y la configuración inicial, y no cambia. Cuando se ejecuta una imagen, se agrega una capa de escritura encima y el resultado es un contenedor.

Se usaron cuatro imágenes. Tres vienen de repositorios públicos y una fue construida en el proyecto:

Imagen	Origen	Tamaño
`gvenzl/oracle-free:23-slim`	Pública	2,84 GB
`banco-internacional-web:1.0.0`	**Propia**	261 MB
`zaproxy/zap-stable:latest`	Pública	3,76 GB
`immauss/openvas:latest`	Pública	7,2 GB

Las imágenes se listan con: `docker images`

2.3 Dockerfile

El Dockerfile es el archivo de texto que describe cómo se construye una imagen. Cada instrucción produce una capa: `FROM` elige la base, `COPY` trae archivos, `RUN` ejecuta comandos durante la construcción, `CMD` define qué se ejecuta al arrancar.

El proyecto usa un Dockerfile de dos etapas para la aplicación web, que es el entregable central de la evaluación. Se detalla en la sección 4.

2.4 Volumen

Un volumen es un espacio de almacenamiento que vive fuera del ciclo de vida del contenedor. Sin volumen, todo lo que se escribe dentro de un contenedor desaparece cuando el contenedor se elimina. Con volumen, los datos sobreviven a los reinicios y las reconstrucciones.

Se usan dos volúmenes nombrados:

- `docker_oracle-data`: guarda los archivos de la base Oracle. Sin él, cada reinicio borraría el esquema y los datos.
- `docker_openvas-data`: guarda los certificados, la base de Greenbone y el feed de vulnerabilidades. Es importante porque el feed pesa cientos de megabytes y descargarlo de nuevo cada vez sería inviable.

Los volúmenes se listan con: `docker volume ls`

2.5 Red en Docker

Una red de Docker conecta contenedores entre sí y les asigna nombres resolubles. Cuando varios contenedores están en la misma red, se encuentran por el nombre del servicio y no por una dirección IP, que puede cambiar.

El proyecto define una red de puente llamada `soc-net`. Los cuatro contenedores están conectados a ella. Gracias a eso, la aplicación se conecta a la base con la cadena `oracle:1521/FREEPDB1` (`oracle` es el nombre del contenedor, no una IP) y los escáneres alcanzan la aplicación en `http://webapp:8088`.

Las redes se listan con: `docker network ls`

y se inspeccionan con: `docker network inspect nombre-red`

2.6 Docker Compose

Docker Compose es la herramienta que permite describir varios contenedores en un archivo YAML y levantarlos con un solo comando. En vez de escribir un `docker run` largo por cada contenedor, se declaran los servicios, las redes y los volúmenes, y se ejecuta `docker compose up`.

El proyecto sigue el diagrama de la evaluación: cada contenedor tiene su propio `docker-compose.yml`, y en la raíz hay uno que los reúne a los cuatro mediante la instrucción `include`. Así se puede levantar todo junto con un comando, o un contenedor suelto cuando conviene.

3. Arquitectura

Cuatro contenedores sobre una red propia. Dos sostienen el servicio y dos lo analizan.

Contenedor	Imagen	Puerto	Rol
`banco-oracle`	`gvenzl/oracle-free:23-slim`	1521	Base de datos, esquema `banco`
`banco-webapp`	`banco-internacional-web:1.0.0`	8088	Aplicación web, imagen propia
`banco-zap`	`zaproxy/zap-stable:latest`	8080	Escáner web
`banco-openvas`	`immauss/openvas:latest`	9392	Escáner de infraestructura

Los cuatro están conectados a la red de puente `soc-net`. La aplicación se conecta a la base con la cadena `oracle:1521/FREEPDB1`: `oracle` es el nombre del contenedor, que la red resuelve a una dirección IP. Los escáneres alcanzan la aplicación en `http://webapp:8088`. Ninguno de los dos necesita una dirección IP fija; el nombre del servicio basta.

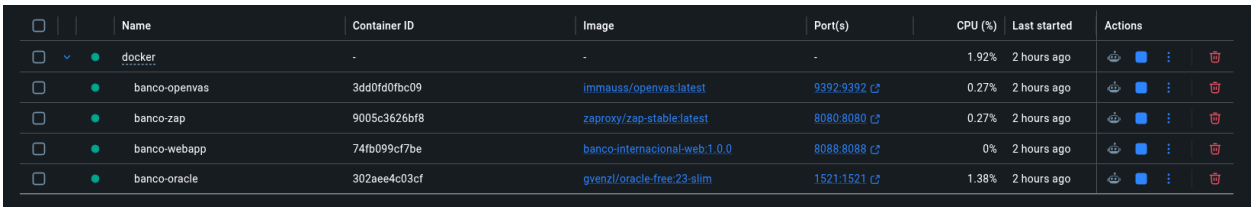


Figura 1. *Los cuatro contenedores en ejecución, con su imagen, estado healthy y puertos expuestos.*

La red se verifica con: `docker network inspect soc-net`


```
[
    {
        "Name": "soc-net",
        "Id": "b0d277273603eb7237b1eb0bce43d3841a6677d7072aa619e730bb51b3823d78",
        "Created": "2026-10-20T21:18:52.737463268Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.enable_ipv4": "true",
            "com.docker.network.enable_ipv6": "false"
```

```
        },
        "Labels": {
            "com.docker.compose.config-hash":
"8bbffe40345d951626dc699134caa1eb830cb03966c04fd534d5a7df2d8a3cfb",
            "com.docker.compose.network": "soc-net",
            "com.docker.compose.project": "docker",
            "com.docker.compose.version": "5.6.0"
        },
        "Containers": {
            "302aee4c03cf95ecaa44a9c916d4ec815aac268a68297810ef58498535b659ca": {
                "Name": "banco-oracle",
                "EndpointID": "a58441dcc97a20ff62cf3ba8a2fb4269dfdc922e083463899cc4d85244e46482",
                "MacAddress": "46:3b:e3:b0:62:b5",
                "IPv4Address": "172.18.0.5/16",
                "IPv6Address": ""
            },
            "3dd0fd0fbc09bb265b890716b7ac923fd2323f6a1c971bd5afb1034a1f00ea87": {
                "Name": "banco-openvas",
                "EndpointID": "4f6488512062fe1c2af2eadca01810786c0292c26206cb530a456fb3caa53a3b",
                "MacAddress": "92:c5:d9:aa:bf:5a",
                "IPv4Address": "172.18.0.4/16",
                "IPv6Address": ""
            },
            "74fb099cf7be52bb744313db29982f0cfa2e0e2cf90ac0379d271728f6bb71da": {
                "Name": "banco-webapp",
                "EndpointID": "95f016ebd03e968204544c3e909072f3991bea59197987e51fad0eaa9e30b7a9",
                "MacAddress": "c2:75:4e:84:f8:9a",
                "IPv4Address": "172.18.0.3/16",
                "IPv6Address": ""
            },
            "9005c3626bf8f1109447cd0b4a62f2f57f327a43a2879da0134e22f68a605fc6": {
                "Name": "banco-zap",
                "EndpointID": "7bc253bfad8b345671b9517958e9edd85d0d8de3576d6dd6688711ff545c6d25",
                "MacAddress": "fa:cd:1c:92:14:f2",
                "IPv4Address": "172.18.0.2/16",
                "IPv6Address": ""
            }
        },
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.18.0.0/16": {
                        "IPsInUse": 7,
                        "DynamicIPsAvailable": 65529
                    }
                }
            }
        }
    }
]
```

4. La imagen propia

La evaluación pide una imagen construida con Dockerfile para la aplicación. Es `banco-internacional-web:1.0.0`, y ocupa 261 MB.

El Dockerfile usa dos etapas. La primera instala las dependencias; la segunda copia solo lo necesario y arma la imagen final. La ventaja es que el resultado no carga las herramientas de compilación ni los archivos temporales de la instalación.

Etapa 1: dependencias. Parte de `node:22-alpine` y ejecuta `npm install --omit=dev`. Al copiar primero solo `package.json` y después el resto, la caché de capas se conserva: mientras no cambie la lista de dependencias, esta etapa no se repite.

```
# syntax=docker/dockerfile:1

# ============================================================
# Banco Internacional - imagen de la aplicacion web
#
# Build en dos etapas: la primera instala las dependencias, la
# segunda solo copia lo necesario. La imagen final corre como
# usuario sin privilegios.
#
# Build:  docker build -t banco-internacional-web .
# ============================================================

# ---------- Etapa 1: dependencias ----------
FROM node:22-alpine AS dependencias

WORKDIR /app

# Las dependencias se copian solas para aprovechar la cache de capas:
# solo se reinstalan cuando cambia package.json.
COPY package.json ./

# Instalacion limpia, solo dependencias de produccion.
RUN npm install --omit=dev --no-audit --no-fund
```

Etapa 2: imagen final. Vuelve a partir de `node:22-alpine`, copia los `node_modules` de la etapa anterior y el código. Agrega las etiquetas `LABEL`, que quedan visibles con `docker inspect`, y define el `HEALTHCHECK` que consulta `/api/health` cada 30 segundos.

```
# ---------- Etapa 2: imagen final ----------
FROM node:22-alpine AS final

# Etiquetas de la imagen, utiles al inspeccionarla con docker inspect.
LABEL org.opencontainers.image.title="Banco Internacional Web" \
      org.opencontainers.image.description="Aplicacion web bancaria del proyecto EV-2" \
      org.opencontainers.image.authors="vis-hoo" \
      org.opencontainers.image.version="1.0.0"

ENV NODE_ENV=production \
    PORT=8088

WORKDIR /app
```

```
# Dependencias ya instaladas, traidas desde la etapa anterior.
COPY --from=dependencias /app/node_modules ./node_modules

# Codigo de la aplicacion.
COPY package.json ./
COPY src ./src

# El usuario node viene en la imagen base; se usa para no correr como root.
RUN chown -R node:node /app
USER node

EXPOSE 8088

# Verificacion interna del contenedor.
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 \
  CMD node -e "require('http').get('http://127.0.0.1:8088/api/health', r => process.exit(r.statusCode
=== 200 ? 0 : 1)).on('error', () => process.exit(1))"

CMD ["node", "src/server.js"]
```

Dos decisiones merecen mención. La imagen corre como el usuario `node`, que viene en la imagen base, y no como `root`, así que si alguien logra ejecutar código dentro del contenedor no lo hace con privilegios de administrador. El `HEALTHCHECK`, por su parte, le da a Docker una forma de saber si la aplicación está viva, y es lo que permite que `docker ps` muestre `healthy` en lugar de solo `Up`.

La imagen se construye con: `docker build -t banco-internacional-web:1.0.0 docker/webapp`

La imagen final se inspecciona con `docker image inspect banco-internacional-web:1.0.0`, que muestra las etiquetas y la configuración declaradas en el Dockerfile.

El resultado del build se comprueba con el listado de imágenes de la Figura 1.

5. La aplicación

Banco Internacional es una aplicación bancaria ficticia con dos roles. Un **cliente** consulta su cuenta y hace depósitos y retiros. Un **ejecutivo** administra clientes, préstamos, movimientos y condiciones de préstamo.

El acceso es simple a propósito: el login entrega usuario y contraseña, y la aplicación abre una sesión que dura lo que dure la visita. Esa simplicidad es parte del ejercicio, porque la sección 8 documenta lo que deja abierto.

Roles y alcance

El servidor decide qué puede hacer cada rol antes de atender la petición. Cada grupo de rutas pasa por un control distinto:

```
app.use('/api', authRoutes);                                          // público
app.use('/api/cliente', requiereSesion, clienteRoutes);              // cualquier sesión
app.use('/api/ejecutivo', requiereRol('ejecutivo'), ejecutivoRoutes); // solo ejecutivo
```

Un cliente que intente llamar a una ruta de ejecutivo recibe una negativa del servidor. La comprobación ocurre en el backend, donde debe estar.

Funcionalidad

Cliente. Ve una sola cuenta, con su saldo, sus movimientos y sus préstamos. Registra depósitos y retiros. La cuenta se consulta con el identificador que quedó en su sesión, así que la consulta siempre devuelve la suya.

Ruta	Método	Qué hace
`/api/cliente/cuenta`	GET	Saldo y datos de la cuenta
`/api/cliente/movimientos`	GET	Historial de movimientos
`/api/cliente/prestamos`	GET	Préstamos vigentes
`/api/cliente/operacion`	POST	Depósito o retiro

Ejecutivo. Tiene control completo sobre las entidades del banco: crear, leer, actualizar y eliminar clientes, préstamos y movimientos.

Ruta	Método	Qué hace
`/api/ejecutivo/clientes`	GET, POST	Lista y crea clientes
`/api/ejecutivo/clientes/:id`	GET, PUT, DELETE	Consulta, actualiza y elimina
`/api/ejecutivo/prestamos`	GET, POST	Lista y crea préstamos
`/api/ejecutivo/prestamos/:id`	PUT, DELETE	Actualiza y elimina
`/api/ejecutivo/movimientos`	GET, POST	Lista y registra movimientos

Modelo de datos

Cinco tablas en el esquema `banco` de Oracle. Un cliente tiene una sola cuenta, relacionadas por clave foránea.

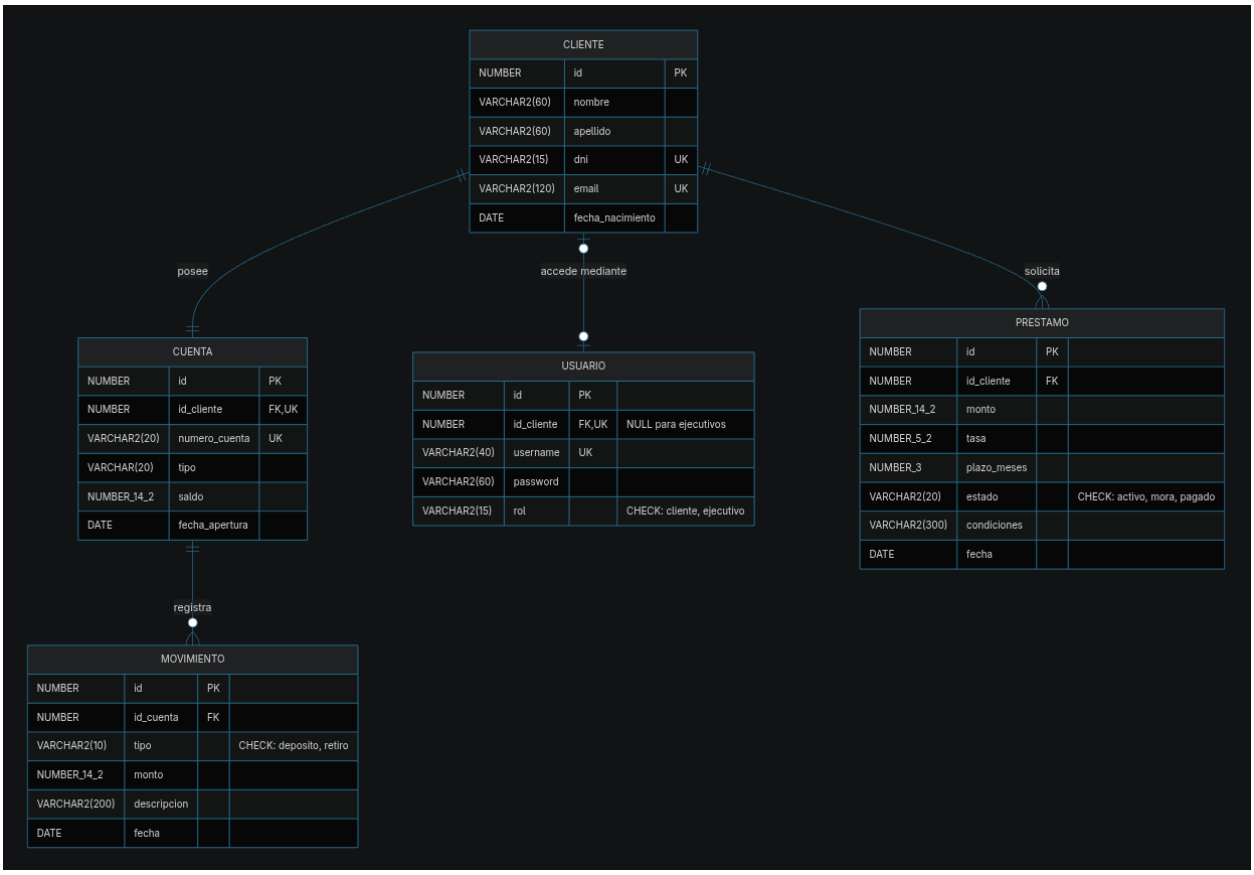


Figura 2. *Las cinco tablas del esquema banco y sus relaciones por clave foránea.*

Capturas

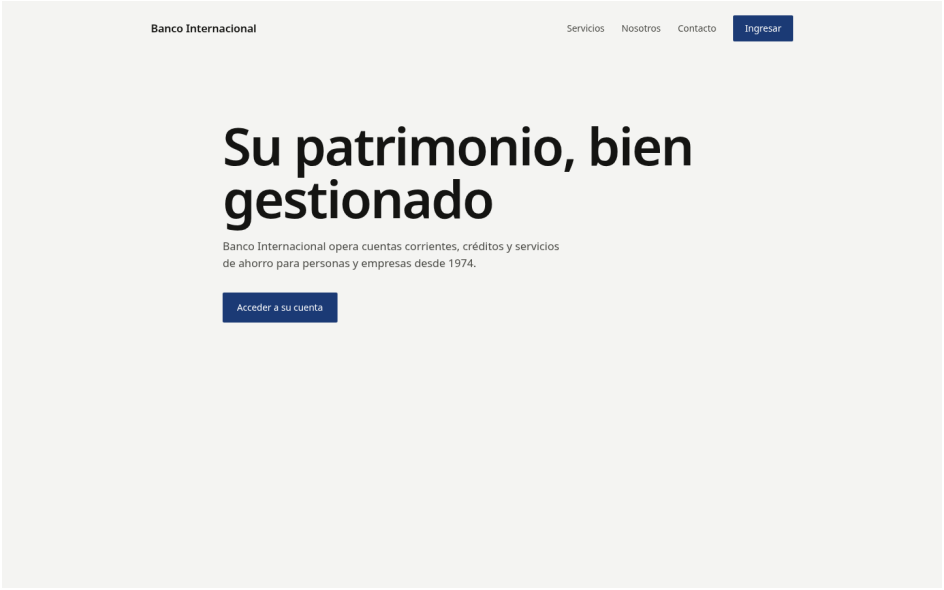


Figura 3. *Página inicial de Banco Internacional.*

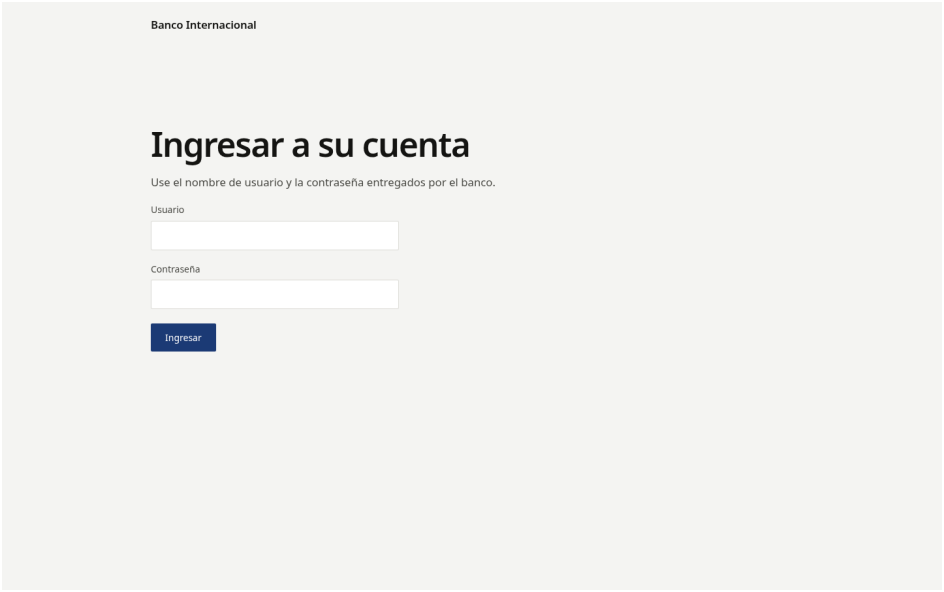


Figura 4. *Formulario de inicio de sesión.*

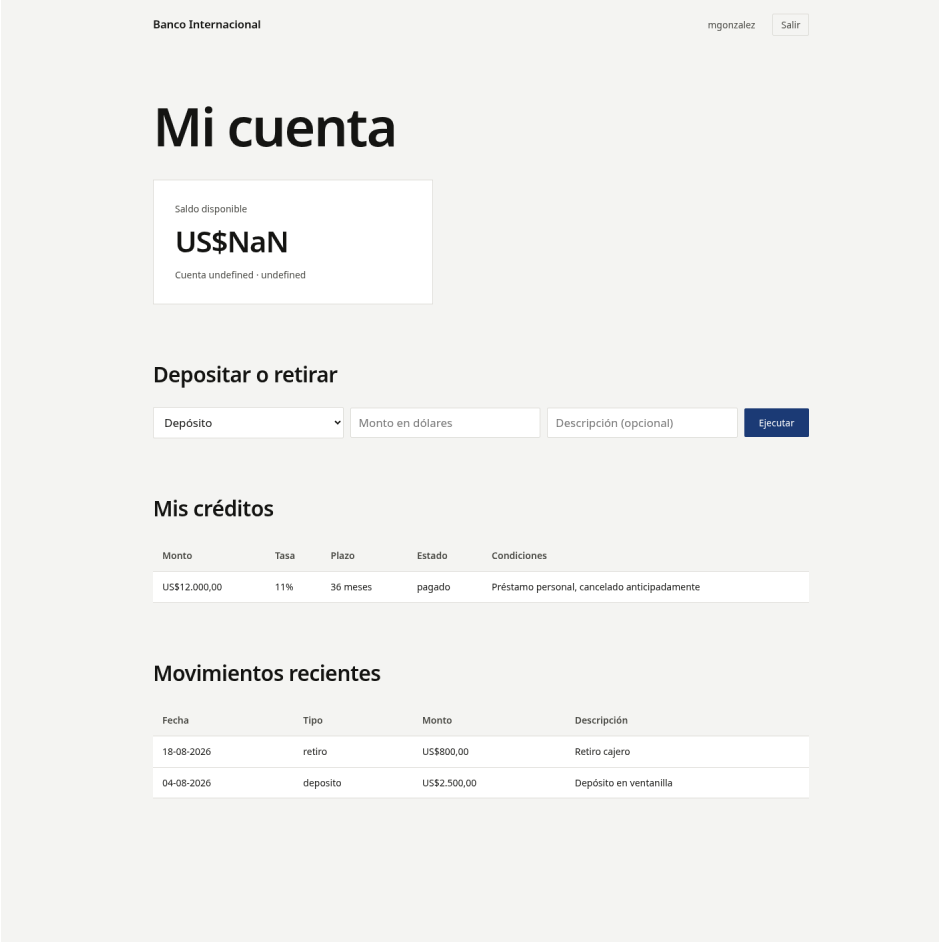


Figura 5. *Panel del cliente, con saldo, préstamos y movimientos.*

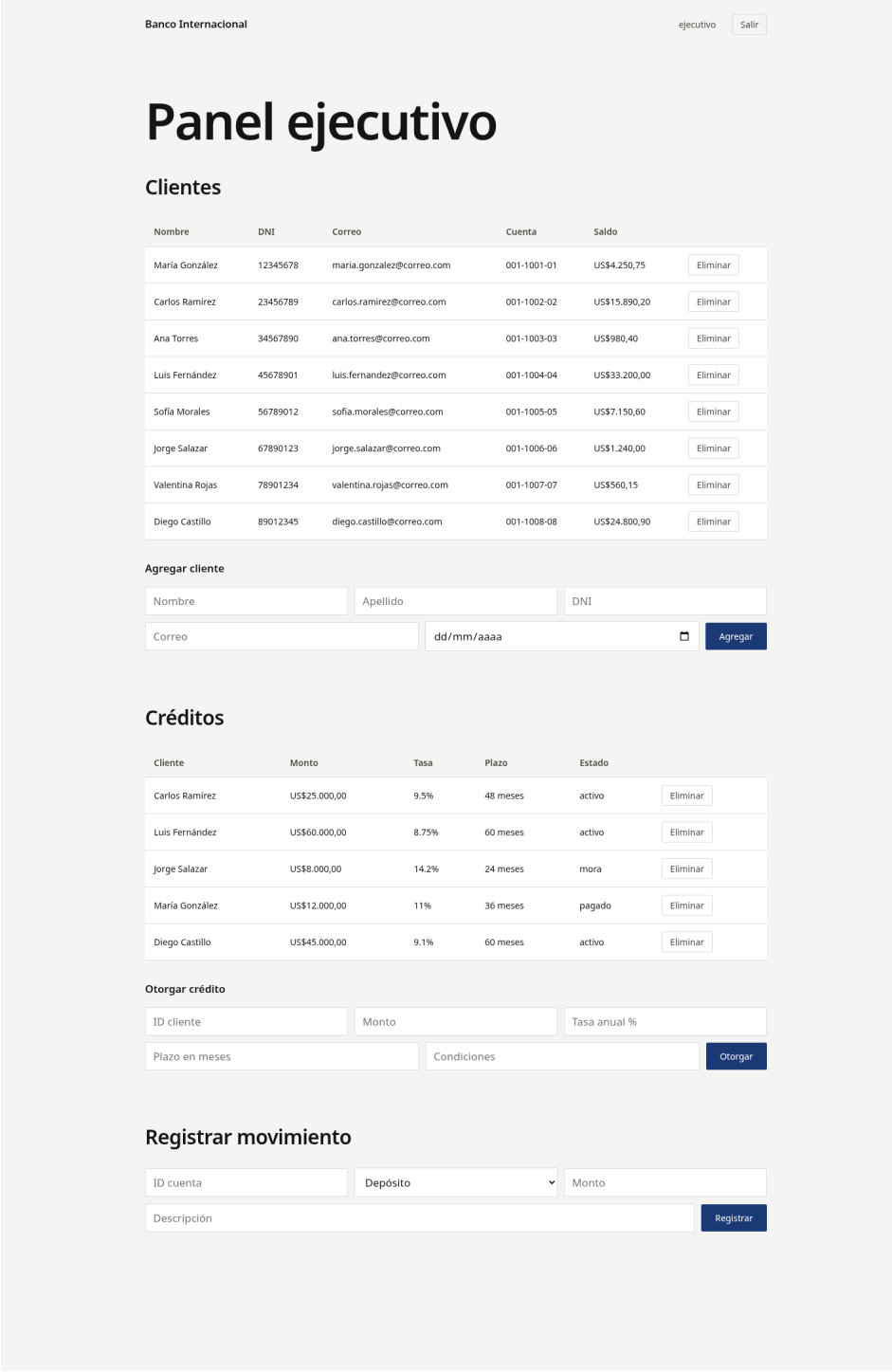


Figura 6. *Panel del ejecutivo, con la lista de clientes.*

6. Escaneo con OWASP ZAP

OWASP ZAP es un escáner de aplicaciones web: navega el sitio, observa las respuestas y, en el modo activo, intenta ataques contra los parámetros que encuentra. Corre en el contenedor `banco-zap` y ataca `http://webapp:8088` desde dentro de la red.

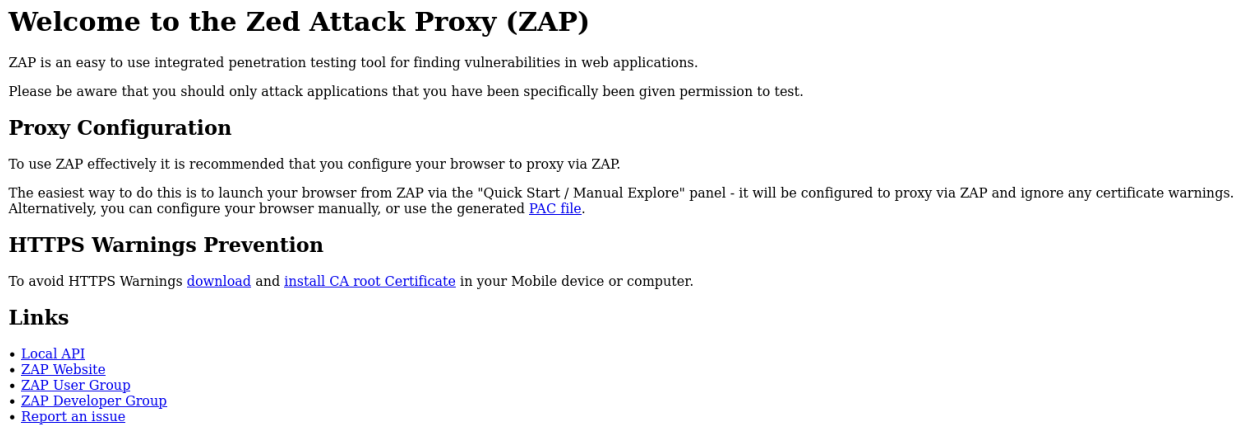


Figura 7. *Página de bienvenida de OWASP ZAP, servida por el contenedor banco-zap en el puerto 8080. Confirma que el escáner web está en ejecución y accesible.*

Se usaron dos escáneres. El primero es pasivo: rastrea y observa, sin atacar. El segundo es activo: además del rastreo, envía cargas diseñadas para provocar fallos.

Escáner 1: pasivo

Rastreó la aplicación, encontró las páginas públicas y el formulario de login, y revisó las respuestas. Reportó **7 tipos de alerta** en 20 instancias.

Riesgo	Alerta	Instancias	CWE
Media	Content Security Policy (CSP) Header Not Set	4	693
Media	Missing Anti-clickjacking Header	4	1021
Media	CSP: Failure to Define Directive with No Fallback	2	693
Baja	Server Leaks Information via "X-Powered-By" Header	5	497
Baja	X-Content-Type-Options Header Missing	5	693
Informativa	Information Disclosure - Sensitive Information in URL	2	598
Informativa	Authentication Request Identified	1	—

El patrón es claro: la aplicación no envía las cabeceras de seguridad que los navegadores esperan. Sin `Content-Security-Policy`, no hay una política que limite qué scripts pueden ejecutarse. Sin `X-Frame-Options`, la página se puede incrustar en un iframe ajeno. Sin `X-Content-Type-Options`, el navegador puede interpretar un archivo como un tipo distinto del declarado. Ninguna abre una brecha por sí sola, pero las tres juntas ensanchan la superficie.

Las dos alertas informativas apuntan a lo mismo desde otro ángulo: el servidor declara que corre Express, y las credenciales viajan en la URL del formulario, donde quedan registradas.

Escáner 2: activo

Repitió el rastreo, procesó **797 solicitudes** y sumó una alerta: *User Agent Fuzzer*, que revela que la aplicación responde distinto ante ciertos agentes de usuario.

Riesgo	Alerta	Instancias	CWE
Media	Content Security Policy (CSP) Header Not Set	4	693
Media	Missing Anti-clickjacking Header	4	1021
Media	CSP: Failure to Define Directive with No Fallback	2	693
Baja	Server Leaks Information via "X-Powered-By" Header	5	497
Baja	X-Content-Type-Options Header Missing	5	693
Informativa	User Agent Fuzzer	5	—
Informativa	Information Disclosure - Sensitive Information in URL	2	598
Informativa	Authentication Request Identified	1	—

El límite del escáner automático

El escaneo activo **no detectó la inyección SQL del inicio de sesión**, y esa vulnerabilidad existe: al enviar la contraseña `' OR '1'='1` la aplicación autentica como la primera fila de la tabla `usuarios`, sin conocer ninguna credencial.

```
$ curl -s -X POST http://127.0.0.1:8088/api/login \
    -H 'Content-Type: application/json' \
    -d '{"username":"ejecutivo","password":"'\'' OR '\''1'\''='\''1"}'
{"ok":true,"rol":"cliente","username":"mgonzalez"}
```

El resultado es doble. El atacante entra sin credenciales, y además entra como un usuario distinto del que pidió: envió `ejecutivo` y la aplicación le respondió `cliente`. La inyección salta la autenticación y devuelve la primera fila que encuentra.

ZAP no la ve porque el login entrega las credenciales en el cuerpo de la petición como JSON, y el conjunto de reglas por defecto no ataca ese formato sin configuración adicional. Un escáner automático cubre lo que ya conoce y deja pasar lo que no. Por eso el hallazgo más grave del trabajo lo encontró la revisión manual, no la herramienta.

Alertas de riesgo medio:


```
{
  "pluginid": "10055",
  "alertRef": "10055-13",
  "alert": "CSP: Failure to Define Directive with No Fallback",
  "name": "CSP: Failure to Define Directive with No Fallback",
  "riskcode": "2",
  "confidence": "3",
  "riskdesc": "Medium (High)",
  "desc": "<p>The Content Security Policy fails to define one of the directives that has no fallback.
Missing/excluding them is the same as allowing anything.</p>",
  "instances":[
    {
      "id": "1",
```

```
      "uri": "http://webapp:8088/robots.txt",
      "nodeName": "http:\/\/webapp:8088\/robots.txt",
      "method": "GET",
      "param": "Content-Security-Policy",
      "attack": "",
      "evidence": "default-src 'none'",
      "otherinfo": "The directive(s): frame-ancestors, form-action is/are among the directives that do
not fallback to default-src."
    },
    {
      "id": "0",
      "uri": "http://webapp:8088/sitemap.xml",
      "nodeName": "http:\/\/webapp:8088\/sitemap.xml",
      "method": "GET",
      "param": "Content-Security-Policy",
      "attack": "",
      "evidence": "default-src 'none'",
      "otherinfo": "The directive(s): frame-ancestors, form-action is/are among the directives that do
not fallback to default-src."
    }
  ],
  "count": "2",
  "systemic": false,
  "solution": "<p>Ensure that your web server, application server, load balancer, etc. is properly
configured to set the Content-Security-Policy header.</p>",
  "otherinfo": "<p>The directive(s): frame-ancestors, form-action is/are among the directives that do
not fallback to default-src.</p>",
  "reference": "<p>https://www.w3.org/TR/CSP/</p><p>https://caniuse.com/
#search=content+security+policy</p><p>https://content-security-policy.com/
</p><p>https://github.com/HtmlUnit/htmlunit-csp</p><p>https://web.dev/articles/csp#resource-
options</p>",
  "cweid": "693",
  "wascid": "15",
  "sourceid": "4"
},
{
  "pluginid": "10038",
  "alertRef": "10038-1",
  "alert": "Content Security Policy (CSP) Header Not Set",
  "name": "Content Security Policy (CSP) Header Not Set",
  "riskcode": "2",
  "confidence": "3",
  "riskdesc": "Medium (High)",
  "desc": "<p>Content Security Policy (CSP) is an added layer of security that helps to detect and
mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks.
These attacks are used for everything from data theft to site defacement or distribution of malware.
CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of
content that browsers should be allowed to load on that page \u2014 covered types are JavaScript, CSS,
HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video
files.</p>",
  "instances":[
    {
      "id": "16",
      "uri": "http://webapp:8088",
      "nodeName": "http:\/\/webapp:8088",
      "method": "GET",
      "param": "",
      "attack": "",
      "evidence": "",
      "otherinfo": ""
    },
    {
```

```
      "id": "11",
      "uri": "http://webapp:8088/",
      "nodeName": "http:\/\/webapp:8088\/",
      "method": "GET",
      "param": "",
      "attack": "",
      "evidence": "",
      "otherinfo": ""
    },
    {
      "id": "10",
      "uri": "http://webapp:8088/login",
      "nodeName": "http:\/\/webapp:8088\/login",
      "method": "GET",
      "param": "",
      "attack": "",
      "evidence": "",
      "otherinfo": ""
    },
    {
      "id": "14",
      "uri": "http://webapp:8088/login?password=ZAP&username=ZAP",
      "nodeName": "http:\/\/webapp:8088\/login (password,username)",
      "method": "GET",
      "param": "",
      "attack": "",
      "evidence": "",
      "otherinfo": ""
    }
  ],
  "count": "4",
  "systemic": false,
  "solution": "<p>Ensure that your web server, application server, load balancer, etc. is configured
to set the Content-Security-Policy header.</p>",
  "otherinfo": "",
  "reference": "<p>https://developer.mozilla.org/en-
US/docs/Web/HTTP/Guides/CSP</p><p>https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Poli
cy_Cheat_Sheet.html</p><p>https://www.w3.org/TR/CSP/</p><p>https://w3c.github.io/webappsec-csp/
</p><p>https://web.dev/articles/csp</p><p>https://caniuse.com/
#feat=contentsecuritypolicy</p><p>https://content-security-policy.com/</p>",
  "cweid": "693",
  "wascid": "15",
  "sourceid": "7"
},
{
  "pluginid": "10020",
  "alertRef": "10020-1",
  "alert": "Missing Anti-clickjacking Header",
  "name": "Missing Anti-clickjacking Header",
  "riskcode": "2",
  "confidence": "2",
  "riskdesc": "Medium (Medium)",
  "desc": "<p>The response does not protect against 'ClickJacking' attacks. It should include either
Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.</p>",
  "instances":[
    {
      "id": "3",
      "uri": "http://webapp:8088",
      "nodeName": "http:\/\/webapp:8088",
      "method": "GET",
      "param": "x-frame-options",
      "attack": "",
```

```
      "evidence": "",
      "otherinfo": ""
    },
    {
      "id": "6",
      "uri": "http://webapp:8088/",
      "nodeName": "http:\/\/webapp:8088\/",
      "method": "GET",
      "param": "x-frame-options",
      "attack": "",
      "evidence": "",
      "otherinfo": ""
    },
    {
      "id": "2",
      "uri": "http://webapp:8088/login",
      "nodeName": "http:\/\/webapp:8088\/login",
      "method": "GET",
      "param": "x-frame-options",
      "attack": "",
      "evidence": "",
      "otherinfo": ""
    },
    {
      "id": "9",
      "uri": "http://webapp:8088/login?password=ZAP&username=ZAP",
      "nodeName": "http:\/\/webapp:8088\/login (password,username)",
      "method": "GET",
      "param": "x-frame-options",
      "attack": "",
      "evidence": "",
      "otherinfo": ""
    }
  ],
  "count": "4",
  "systemic": false,
  "solution": "<p>Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP
headers. Ensure one of them is set on all web pages returned by your site/app.</p><p>If you expect the
page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use
SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively
consider implementing Content Security Policy's \"frame-ancestors\" directive.</p>",
  "otherinfo": "",
  "reference": "<p>https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-
Options</p>",
  "cweid": "1021",
  "wascid": "15",
  "sourceid": "7"
}
```

7. Escaneo con OpenVAS

OpenVAS es un escáner de infraestructura. Donde ZAP mira el código y los parámetros, OpenVAS mira la red: qué puertos están abiertos, qué servicios corren detrás y qué vulnerabilidades conocidas les corresponden. Corre en el contenedor `banco-openvas` y su objetivo es `webapp`, alcanzado por nombre dentro de `soc-net`.

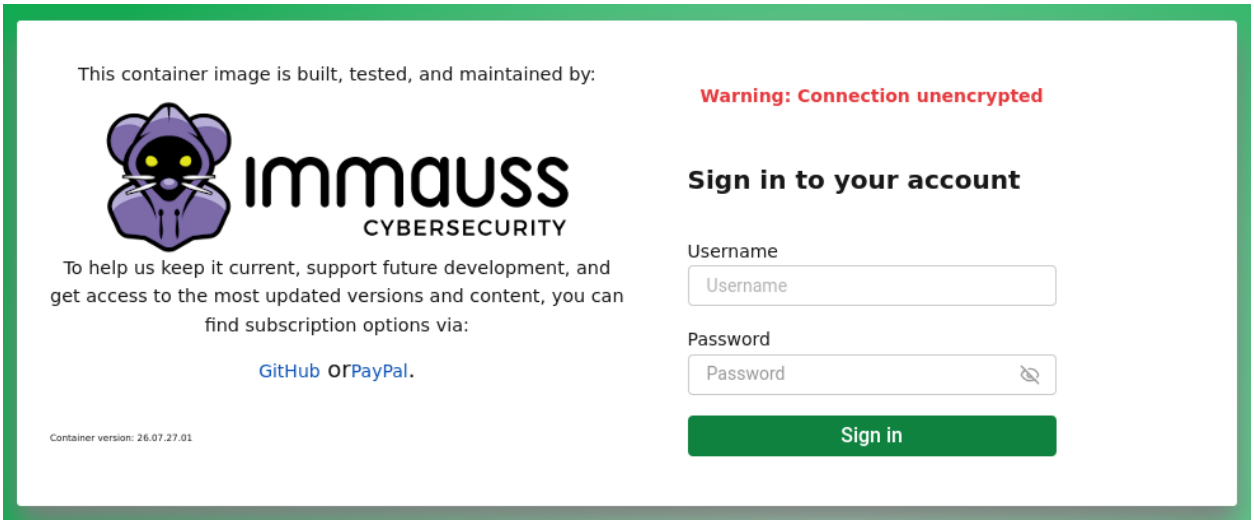


Figura 8. *Interfaz web de OpenVAS en el puerto 9392, servida por el contenedor banco-openvas. El aviso en rojo ("Connection unencrypted") indica que el acceso a la consola es por HTTP, sin cifrar.*

Se maneja por GMP (Greenbone Management Protocol) sobre el socket interno de `gvmd`, con `gvm-cli` ejecutado como el usuario `gvm`. La automatización se resuelve con un script que crea el objetivo, crea la tarea y la inicia:

```
# scripts/run-openvas-scans.sh (extracto)
TARGET_ID=$(gvm-cli socket --gmp-username admin --gmp-password admin \
  --xml "<create_target><name>webapp</name><hosts>webapp</hosts></create_target>" \
  | grep -oP '(?<=id=")[^"]+')

gvm-cli socket --gmp-username admin --gmp-password admin \
  --xml "<create_task><name>escaneo-completo</name><target id=\"$TARGET_ID\"/></create_task>"

gvm-cli socket --gmp-username admin --gmp-password admin \
  --xml "<start_task task_id=\"$TASK_ID\"/>"
```

Escáner 1: reconocimiento

Perfil **Discovery** sobre los 5836 puertos TCP asignados por IANA. Duró 72 segundos y devolvió 8 hallazgos.

Nivel	Sev.	Puerto	Hallazgo
Baja	2.1	general/icmp	ICMP Timestamp Reply Information Disclosure
Informativa	0.0	8088/tcp	Express Detection (HTTP)
Informativa	0.0	8088/tcp	Services
Informativa	0.0	general/tcp	Hostname Determination Reporting
Informativa	0.0	general/tcp	OS Detection Consolidation and Reporting

Informativa	0.0	general/tcp	Traceroute

El escáner encontró el puerto 8088 abierto y lo identificó como Express sobre HTTP. Para quien busca atacar, eso acota la superficie y nombra la tecnología. Los demás hallazgos registran cómo OpenVAS llegó hasta ahí y el ICMP timestamp, que revela el reloj del equipo.

Escáner 2: completo

Perfil **Full and fast** sobre todos los TCP y los 100 UDP más comunes de Nmap. Duró 6 minutos 11 segundos y devolvió 10 hallazgos, entre ellos uno de severidad media.

Nivel	Sev.	Puerto	Hallazgo
Media	4.8	8088/tcp	Cleartext Transmission of Sensitive Information via HTTP
Informativa	0.0	8088/tcp	Express Detection (HTTP)
Informativa	0.0	8088/tcp	Allowed HTTP Methods Enumeration
Informativa	0.0	8088/tcp	HTTP Security Headers Detection
Informativa	0.0	general/CPE-T	CPE Inventory
Informativa	0.0	general/tcp	Hostname Determination Reporting

El hallazgo de severidad media es el que importa. OpenVAS identificó el campo `password` del formulario de login y confirmó que viaja por HTTP sin cifrar. El reporte nombra el campo y las dos rutas donde aparece, `/Login` y `/login`.


```
{
  "name": "Cleartext Transmission of Sensitive Information via HTTP",
  "owner": {
    "name": "admin"
  },
  "modification_time": "2026-10-20T22:01:55Z",
  "comment": "",
  "creation_time": "2026-10-20T22:01:55Z",
  "host": {
    "#text": "172.18.0.4",
    "asset": "",
    "hostname": "webapp"
  },
  "port": "8088/tcp",
  "nvt": {
    "type": "nvt",
    "name": "Cleartext Transmission of Sensitive Information via HTTP",
    "family": "Web application abuses",
    "cvss_base": 4.8,
    "severities": {
      "severity": {
        "origin": "",
        "date": "2018-04-16T13:31:23Z",
        "score": 4.8,
        "value": "AV:A/AC:L/Au:N/C:P/I:P/A:N"
      }
    },
    "tags": "cvss_base_vector=AV:A/AC:L/Au:N/C:P/I:P/A:N|summary=The host / application transmits
sensitive information (username, passwords) in\n  cleartext via HTTP.|insight=|affected=Hosts /
applications which doesn&apos;t enforce the transmission of sensitive data via an\n  encrypted SSL/TLS
```

```
connection.|impact=An attacker could use this situation to compromise or eavesdrop on the\n  HTTP
communication between the client and the server using a man-in-the-middle attack to get access to\n
sensitive data like usernames or passwords.|solution=Enforce the transmission of sensitive data via an
encrypted SSL/TLS connection.\n  Additionally make sure the host / application is redirecting all
users to the secured SSL/TLS connection before\n  allowing to input sensitive data into the mentioned
functions.|vuldetect=Evaluate previous collected information and check if the host / application is
not\n  enforcing the transmission of sensitive data via an encrypted SSL/TLS connection.\n\n  The
script is currently checking the following:\n\n  - HTTP Basic Authentication (Basic Auth)\n\n  - HTTP
Forms (e.g. Login) with input field of type &apos;password&apos;|solution_type=Workaround",
    "solution": "Enforce the transmission of sensitive data via an encrypted SSL/TLS connection.\n
Additionally make sure the host / application is redirecting all users to the secured SSL/TLS
connection before\n  allowing to input sensitive data into the mentioned functions.",
    "refs": {
      "ref": [
        "",
        "",
        ""
      ]
    }
  },
  "scan_nvt_version": "2023-09-07T05:05:21Z",
  "threat": "Medium",
  "severity": 4.8,
  "qod": {
    "value": 80,
    "type": ""
  },
  "description": "The following input fields were identified (URL:input name):
\n\nhttp://webapp:8088/Login:password\nhttp://webapp:8088/login:password",
  "original_threat": "Medium",
  "original_severity": 4.8,
  "compliance": "undefined"
}
```


ZAP reporta la misma debilidad por otro camino: la llama *Information Disclosure - Sensitive Information in URL* y la encuentra en la URL. OpenVAS la ve en el transporte. Dos herramientas independientes, dos métodos distintos, el mismo problema.

8. Tabla de vulnerabilidades

Los trece hallazgos del trabajo, ordenados por severidad. La columna *Detectado por* separa lo que encontró un escáner de lo que encontró la revisión manual del código.

#	Vulnerabilidad	Severidad	Detectado por	Contenedor
1	Inyección SQL en el login (salta la autenticación)	Alta	**Revisión manual**	webapp
2	Contraseñas en texto plano en la tabla `usuarios`	Alta	**Revisión manual**	oracle
3	Credenciales enviadas por HTTP sin cifrar	Media	OpenVAS (4.8) y ZAP	webapp
4	Credenciales en la URL del formulario (`GET`)	Media	ZAP	webapp
5	Cookie de sesión sin `httpOnly` ni `secure`	Media	**Revisión manual**	webapp
6	Sin límite de intentos en el login	Media	**Revisión manual**	webapp
7	`Content-Security-Policy` ausente	Media	ZAP (CWE 693)	webapp
8	`X-Frame-Options` ausente (clickjacking)	Media	ZAP (CWE 1021)	webapp
9	Secreto de sesión escrito en el código	Media	**Revisión manual**	webapp
10	ICMP Timestamp Reply Information Disclosure	Baja	OpenVAS (2.1)	webapp
11	`X-Content-Type-Options` ausente	Baja	ZAP (CWE 693)	webapp
12	Cabecera `X-Powered-By` revela Express	Baja	ZAP (CWE 497)	webapp
13	Portafolio de servicios expuesto (Express en 8088)	Informativa	OpenVAS	webapp

El reparto merece un comentario. Los escáneres automáticos, entre los dos y con cuatro corridas, detectaron ocho hallazgos: cinco solo ZAP, dos solo OpenVAS y uno los dos. La revisión manual del código aportó los cinco restantes. La inyección SQL, la única de severidad alta que permite entrar sin credenciales, no está en ninguno de los cuatro reportes.

Los dos escáneres coinciden en un punto: las credenciales viajan sin protección. ZAP lo ve en la URL y OpenVAS en el transporte. Dos herramientas con métodos distintos llegan al mismo hallazgo, y eso lo hace más sólido que si lo reportara una sola.

9. Plan de mitigación

Cada hallazgo se corrige en la capa que le corresponde, no donde se detectó. Que ZAP reporte una cabecera ausente no significa que el problema esté en el escáner: está en la configuración del servidor. El plan completo está en `docs/mitigacion.md`; aquí va lo esencial y el orden en que conviene trabajarlo.

Las correcciones se agrupan en dos bloques. El primero resuelve el riesgo real; el segundo endurece la superficie.

Bloque 1: riesgo real.

1. **Inyección SQL en el login.** La consulta se arma concatenando los datos del formulario. La corrección es usar enlaces en la consulta, como ya se hace en el resto de las rutas: `WHERE username = :usuario AND password_hash = :hash`. Es un cambio de una línea y elimina el problema de raíz: el motor trata los valores como datos, no como instrucciones.
2. **Contraseñas en texto plano.** Hoy la tabla `usuarios` guarda las contraseñas legibles. Cualquiera con acceso de lectura, o con la inyección del punto anterior, obtiene todas las credenciales de golpe. Se corrige guardando un hash con `bcrypt` y comparando con `bcrypt.compare` en el login.
3. **Sin límite de intentos.** Se puede probar contraseñas sin freno. Se corrige limitando los intentos por usuario y por origen con `express-rate-limit`, más un bloqueo temporal tras varios fallos.
4. **Cookie de sesión desprotegida.** La cookie se emite con `httpOnly: false` y el secreto de firma está escrito en el código. El primer valor permite que un script inyectado lea la sesión; el segundo permite falsificarla. Se corrige con `httpOnly: true`, `secure: true`, `sameSite: 'strict'` y el secreto en una variable de entorno.

Bloque 2: endurecimiento.

5. **Cabeceras de seguridad.** Se agrega `helmet` al arranque de Express, que trae `Content-Security-Policy`, `X-Frame-Options` y `X-Content-Type-Options` con valores por defecto razonables.
6. **Fuga de información.** Se desactiva la cabecera `X-Powered-By` (`helmet` ya lo hace) y el formulario de login pasa de `GET` a `POST`, para que las credenciales no queden en la URL.

#	Vulnerabilidad	Impacto	Esfuerzo	Prioridad
1	Inyección SQL en login	Alto	Bajo	Inmediata
2	Contraseñas en texto plano	Alto	Medio	Inmediata
3	Sin límite de intentos	Medio	Bajo	Alta
4	Cookie sin `httpOnly`/`secure`	Alto	Bajo	Alta
5	Cabeceras de seguridad ausentes	Medio	Bajo	Media
6	Fuga en cabecera y URL	Bajo	Bajo	Media

Las cuatro primeras se corrigen en una misma sesión de trabajo y resuelven el grueso del riesgo. El punto 4 merece una nota: además de proteger la cookie, sostiene el aislamiento entre clientes. Hoy ese aislamiento

depende de que el identificador de sesión sea de fiar; si la sesión se puede manipular, la frontera entre una cuenta y otra se cae.

10. Conclusiones

El trabajo montó una infraestructura de cuatro contenedores sobre una red propia y la analizó con dos herramientas. Las dos partes funcionan: los contenedores se comunican entre sí por nombre, la aplicación corre desde una imagen construida en el proyecto, y los dos escáneres alcanzan la aplicación desde dentro de la misma red.

La red es lo que hace posible el resto. Sin `soc-net`, la aplicación tendría que buscar la base por una dirección IP que cambia, y los escáneres no tendrían a quién apuntar. Los nombres de servicio convierten una configuración frágil en una estable.

Una imagen propia también exige decisiones que van más allá de empaquetar. El Dockerfile de dos etapas, el usuario sin privilegios y el `HEALTHCHECK` afectan a la seguridad y a la operación. Que el contenedor corra como `node` y no como `root` no cambia nada mientras todo va bien, y lo cambia todo cuando algo va mal.

Lo más importante que quedó del análisis es que el escáner automático no encontró el fallo más grave. ZAP reportó cabeceras ausentes y fugas de información, todas reales, todas de severidad media o baja. La inyección SQL del login, que permite entrar sin credenciales, no apareció en el reporte y la encontró la revisión manual del código. Un escáner prueba lo que sabe probar, y por eso se configura y se complementa, no se delega.

Las herramientas de análisis resultan útiles como complemento, pero no sustituyen la intervención de personal calificado para revisar el código, validar las hipótesis y detectar fallos lógicos. La combinación de escáneres automatizados y revisión manual es la que permite obtener un diagnóstico completo y fiable.

El plan de mitigación cierra los hallazgos en dos bloques. Las cuatro correcciones del primero son pequeñas y resuelven el grueso del riesgo: arreglar la inyección y las contraseñas rinde más que sumar veinte cabeceras.

Referencias

Greenbone Networks. (2026). *Greenbone Management Protocol (GMP)*. https://greenbone.github.io/docs/latest/

OWASP Foundation. (2026). *OWASP ZAP*. https://www.zaproxy.org/docs/

Docker Inc. (2026). *Docker documentation*. https://docs.docker.com/

Oracle. (2026). *Node.js and Oracle Database*. https://node-oracledb.readthedocs.io/

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/